CE6014
Object Oriented Software Design

ONLINE TUTORING SYSTEM

Using UML · Composition & Interfaces

Name : Suryaprakash Yadav
Enrollment No. : 202303103510411
Division : 7D

REAL-WORLD SCENARIO & SYSTEM OVERVIEW



Students today want expert help beyond the classroom, but finding the right tutor, fixing a time and paying for sessions is still spread across calls and chats. An **Online Tutoring System** brings tutors, students, courses and live sessions onto one platform. It organises course content, schedules sessions, tracks learning progress and handles payments securely.

OBJECT ABSTRACTION MIRRORS REAL-WORLD LEARNING:

- A Course contains Modules; Modules contain Lessons.
- Tutors and Students meet in scheduled Tutoring Sessions.
- Every payment mode plugs in through one Payable interface.

PROBLEM STATEMENT & CORE OBJECTIVES

PROBLEM STATEMENT

Running online learning well means keeping course content organised, matching students with tutors at the right time, and accepting payments in different ways, without the code breaking every time a new feature or payment method is added.

KEY OBJECTIVES

Structured Course Content
A Course owns its Modules and Lessons, so content always stays complete and consistent.

Flexible Session Scheduling
Anything bookable follows one Schedulable contract for booking and cancelling slots.

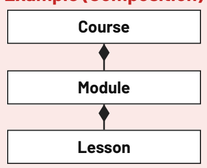
Pluggable Payments
New modes like Card or UPI just implement Payable, with no change to existing code.

COMPOSITION AND INTERFACES

COMPOSITION (Strong Ownership)

- ✓ Strong whole-part relationship
- ✓ Part depends on its whole
- ✓ Shared lifecycle
- ✓ Cascade deletion
- ✓ No orphan modules or lessons

Example (Composition)

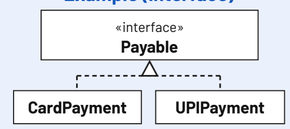


If a Course is deleted, all its Modules and Lessons are deleted too.

INTERFACES (Contract-Based)

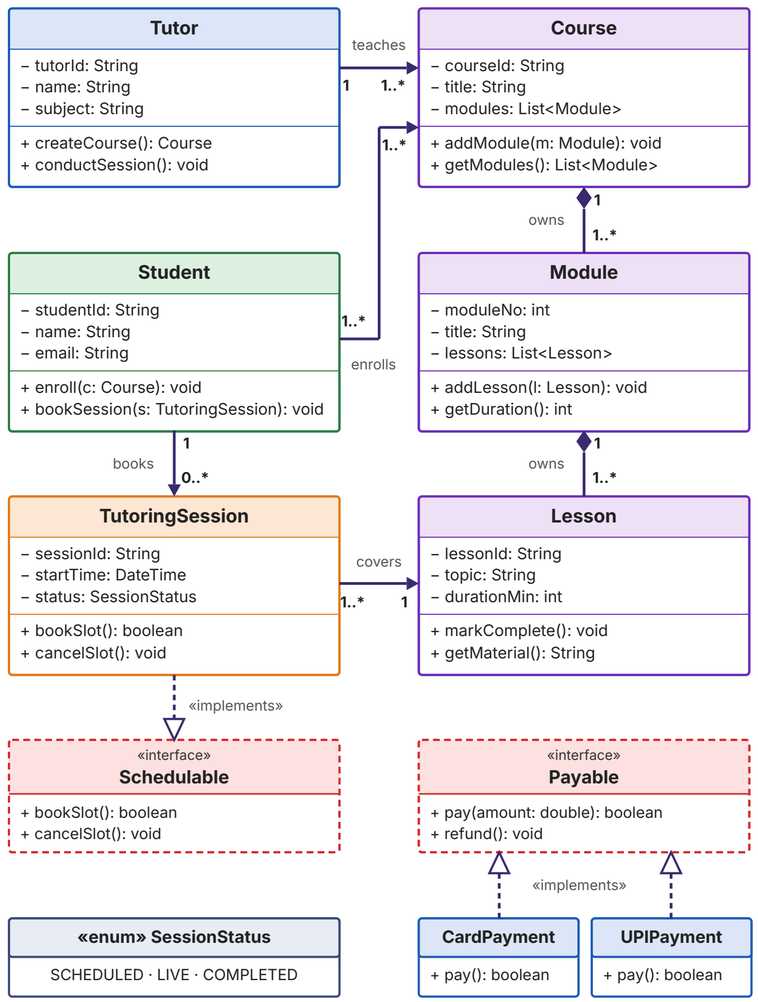
- ✓ Contract of methods only
- ✓ No implementation inside
- ✓ Each class implements its own way
- ✓ Code depends on the contract
- ✓ Swap classes without changes

Example (Interface)



Checkout just calls pay(); adding WalletPayment needs no change elsewhere.

DETAILED UML CLASS DIAGRAM



UML NOTATION LEGEND

- **Composition** — Part lives and dies with its whole.
- **Realization** — Class implements an interface.
- **Association** — Independent objects linked.
- **1** One
- **0..*** Zero or more
- **1..*** One or more
- **« »** Stereotype

STRUCTURAL & RESPONSIBILITY MATRIX

Source Class	Relationship	Target Class	Multiplicity	Lifecycle & Purpose
Course	◆ Composition	Module	**1 ◆— 1..***	Course owns its Modules. Deleting a Course deletes them.
Module	◆ Composition	Lesson	**1 ◆— 1..***	Lessons exist only inside their Module.
TutoringSession	⇢ Realization	«Schedulable»	**implements**	Session gives its own bookSlot() and cancelSlot().
Card / UPI Payment	⇢ Realization	«Payable»	**implements**	Each mode implements pay() and refund() its own way.

OUR OOD CORE CONCEPT

COMPOSITION

- Strong ownership between parent and child objects.
- Child cannot exist without its parent.
- Shared lifecycle and cascade deletion.
- Used for: Course → Module, Module → Lesson.

INTERFACES

- A contract of methods with no body.
- Classes implement it in their own way.
- Callers depend on the contract, not the class.
- Used for: Schedulable, Payable (Card, UPI).

SOFTWARE QUALITY ASSURANCE (QA) & OOD IMPACT

Lifecycle Safety
Composition prevents orphan modules and lessons, so course data stays consistent.

High Cohesion
Each class has one clear role: Course organises, Session schedules, Payment pays.

Extensibility
A new payment mode or bookable item only needs to implement an interface.

Testability
Interfaces allow mock payments and sessions, making unit tests simple and isolated.

FUTURE SCOPE

AI Tutor Matching
- Suggest the best tutor using subject, rating and past results.
- Personalised learning paths.

Live Whiteboard & Recording
- Shared whiteboard inside sessions.
- Recordings saved per Lesson for revision.

Mobile App & Analytics
- Android and iOS apps.
- Progress dashboards and attendance insights.

CONCLUSION

By applying **Composition** and **Interfaces**, the Online Tutoring System keeps course content safely owned and consistent, while scheduling and payments stay flexible through clear contracts. This gives a clean, testable and easily extendable foundation for real-world e-learning.